

Segment 4: Integrity, Security and Relational Database Design

INTEGRITY CONSTRAINTS

Integrity constraints are rules that ensure the correctness, validity, and consistency of data stored in a database. They help a DBMS reflect real-world rules so that invalid data cannot enter the system. Common integrity constraints include **domain constraint**, **referential integrity**, **assertions**, and **triggers**.

Domain Constraint

A domain constraint restricts the permissible values in a column. It ensures that each attribute contains values from an acceptable domain such as a valid range, valid set of strings, correct format, or appropriate data type.

Real-life meaning

- Age must be between 1 and 120
- Gender must be 'M' or 'F'
- Salary must be positive
- Semester must be from a fixed list ('Spring', 'Summer', 'Fall')

SQL Example

```
CREATE TABLE Student (  
    sid NUMBER PRIMARY KEY,  
    sname VARCHAR2(30),  
    age NUMBER CHECK (age BETWEEN 18 AND 40),  
    gender VARCHAR2(1) CHECK (gender IN ('M','F'))  
);
```

This prevents invalid values such as age = -5 or gender = 'X'.

Referential Integrity

Referential integrity ensures that a foreign key in one table must match an existing primary key in another table. It maintains consistency among related tables.

Real-life meaning

- A student must belong to an existing department
- A prescription must refer to an existing patient and doctor
- An employee cannot be assigned to a non-existent branch

SQL Example

```
CREATE TABLE Department (  
    did NUMBER PRIMARY KEY,  
    dname VARCHAR2(20)  
);
```

```
CREATE TABLE Employee (  
    eid NUMBER PRIMARY KEY,  
    ename VARCHAR2(20),  
    did NUMBER REFERENCES Department(did)  
);
```

If no department with did = 50 exists, inserting an employee in department 50 is not allowed.

Assertions (Conceptual Constraint)

An assertion specifies a condition that must always remain true across the entire database.

Real-life meaning

- An instructor cannot teach more than two courses in the same semester
- Total students in a semester must not exceed 500
- No patient can visit two doctors at the same time

Example

Instructor(iid, iname, designation, salary), Teaches(iid, cid, semester)

“Ensure that an instructor cannot teach more than two courses in the same semester.”

```
CREATE ASSERTION MaxTwoCourses  
CHECK (  
    NOT EXISTS (  
        SELECT iid, semester  
        FROM Teaches  
        GROUP BY iid, semester  
        HAVING COUNT(cid) > 2  
    )  
);
```

```
)  
);
```

Triggers

A trigger is a stored program that executes automatically in response to an INSERT, UPDATE, or DELETE operation.

Triggers are used to enforce complex integrity rules that cannot be expressed using simple constraints.

How triggers ensure integrity

- Prevent invalid data entry
- Log unauthorized modifications
- Stop duplicate bookings or double appointments
- Implement business rules such as minimum balance enforcement

Example: prevent entering negative salary

```
CREATE OR REPLACE TRIGGER salary_check  
BEFORE INSERT OR UPDATE ON Instructor  
FOR EACH ROW  
BEGIN  
    IF :NEW.salary < 0 THEN  
        RAISE_APPLICATION_ERROR(-20001,'Salary cannot be negative');  
    END IF;  
END;  
/
```

Hospital Example: Prevent a patient from visiting two doctors at the same time

Schema:

Patient(pid, pname, age, gender)

Doctor(did, dname, specialization)

Prescribe(pid, did, drugid, visit_time)

Oracle Trigger

```
CREATE TRIGGER visit_check  
BEFORE INSERT ON Prescribe  
FOR EACH ROW  
BEGIN  
    IF EXISTS (SELECT 1 FROM Prescribe  
        WHERE pid = :NEW.pid  
        AND visit_time = :NEW.visit_time) THEN
```

```
RAISE_APPLICATION_ERROR(-20002,'Patient already assigned');  
END IF;  
END;  
/
```

AUTHENTICATION vs AUTHORIZATION

Authentication

Authentication confirms the identity of a user.
Example: username/password, OTP, biometric check.

Authorization

Authorization determines what actions the authenticated user is allowed to perform.

Difference (Exam Standard Answer)

Authentication verifies *who* the user is, whereas authorization verifies *what* the user is permitted to do after successful authentication.

PRIVILEGES

A **privilege** is a permission granted to a user to perform specific operations in the database. Privileges are **two main types**:

1. **Object (Data) Privileges** – operate on **table or view data**.
 2. **System (Database) Privileges** – operate on **database structure or schema objects**.
-

1. Object (Data) Privileges

Common object privileges: **SELECT, INSERT, UPDATE, DELETE**.

Schema Example:

```
CREATE TABLE Student (  
    sid NUMBER PRIMARY KEY,  
    sname VARCHAR2(30),  
    age NUMBER,  
    dept VARCHAR2(10)  
);
```

```
INSERT INTO Student VALUES (1, 'Alice', 20, 'CSE');
```

```
INSERT INTO Student VALUES (2, 'Bob', 21, 'EEE');
INSERT INTO Student VALUES (3, 'Charlie', 22, 'CSE');
```

Examples:

- **SELECT (Read)**

```
GRANT SELECT ON Student TO userA;
```

```
-- userA can run:
```

```
SELECT * FROM Student;
```

```
SELECT sname, dept FROM Student WHERE dept = 'CSE';
```

- **INSERT (Add data)**

```
GRANT INSERT ON Student TO userA;
```

```
-- userA adds a new student
```

```
INSERT INTO Student (sid, sname, age, dept) VALUES (4, 'David', 23, 'EEE');
```

- **UPDATE (Modify data)**

```
GRANT UPDATE(age) ON Student TO userA; -- specific column
```

```
UPDATE Student SET age = 24 WHERE sid = 2;
```

```
GRANT UPDATE ON Student TO userB; -- all columns
```

```
UPDATE Student SET sname='Eve', dept='ME' WHERE sid=3;
```

- **DELETE (Remove data)**

```
GRANT DELETE ON Student TO userA;
```

```
-- userA deletes a student
```

```
DELETE FROM Student WHERE sid = 4;
```

- **Using a VIEW for SELECT**

```
CREATE VIEW student_view AS
```

```
SELECT sid, sname, age
```

```
FROM Student
```

```
WHERE dept = 'CSE';
```

```
GRANT SELECT ON student_view TO teacher;
```

2. System (Database) Privileges

System privileges allow control over **tables, views, indexes, and other schema objects**.

Common system privileges: CREATE TABLE, DROP TABLE, ALTER TABLE, CREATE VIEW, CREATE/DROP INDEX

Examples:

- **CREATE TABLE**

GRANT CREATE TABLE TO developer;

```
CREATE TABLE Course (  
  cid NUMBER PRIMARY KEY,  
  cname VARCHAR2(30),  
  credit NUMBER  
);
```

- **ALTER TABLE**

GRANT ALTER ON Student TO db_assistant;

```
-- Add a new column  
ALTER TABLE Student ADD email VARCHAR2(50);
```

```
-- Modify column datatype  
ALTER TABLE Student MODIFY age NUMBER(3);
```

- **DROP TABLE**

GRANT DROP TABLE TO manager;

DROP TABLE Course;

- **CREATE INDEX**

GRANT CREATE INDEX ON Student TO developer;

CREATE INDEX idx_student_dept ON Student(dept);

- **CREATE VIEW**

GRANT CREATE VIEW TO analyst;

```
CREATE VIEW cse_students AS  
SELECT sid, sname  
FROM Student  
WHERE dept='CSE';
```

- **DROP INDEX**

```
GRANT DROP INDEX ON Student TO developer;
```

```
DROP INDEX idx_student_dept;
```

Explanation

- **Object privileges** control **what users can do with table data.**
 - **System privileges** control **how users can modify or create database objects.**
 - Privileges can be **granted** or **revoked** as needed to maintain security.
-

ROLES

A role is a named group of privileges that can be granted to users.
Roles simplify privilege management in large organizations.

Why roles are needed

- To avoid granting privileges to each user individually
- To ensure consistent access control
- To improve security
- To manage departments or job positions easily

Example: Hospital System

```
CREATE ROLE doctor_role;
```

```
GRANT SELECT ON Patient TO doctor_role;  
GRANT INSERT, SELECT ON Prescribe TO doctor_role;
```

```
GRANT doctor_role TO dr_rahim;
```

Revoking and Cascading

```
REVOKE doctor_role FROM dr_rahim;
```

Privileges obtained through a role disappear when the role is revoked.

AUDIT TRAILS

Audit trails are logs that record who performed what operation and when. They help monitor unauthorized access and changes.

Used for

- Security tracking
- Compliance
- Fraud detection
- Accountability

Simple Oracle Example

AUDIT INSERT, UPDATE, DELETE ON Patient;

This creates automatic logs when these actions occur.

ENCRYPTION AND DECRYPTION

Encryption

The process of converting readable data into unreadable form to protect confidentiality.

Decryption

The reverse process — restoring readable data from an encrypted form.

Purpose

- Protect passwords
- Secure medical and banking data
- Prevent unauthorized access

Simple conceptual example:

Plain text:

HELLO

Encrypted:

JFNQ9X#23 (example)

DECOMPOSITION

Decomposition breaks a large relation into smaller, well-structured relations to reduce redundancy and anomalies.

Goals of decomposition:

- Lossless-join
- Dependency preservation
- Removing update anomalies
- Logical clarity

Example

Original table:

Employee(eid, ename, dept_name, dept_location)

Decomposition:

Department(dept_name, dept_location)

Employee(eid, ename, dept_name)

This avoids repeating dept_location for every employee.

COMMON EXAM QUESTIONS AND MODEL ANSWERS

1. What is the difference between authentication and authorization?

- **Authentication** is the process of verifying the identity of a user or system. It answers the question: *“Who is trying to access the database?”*
Example: Username/password login, biometric verification, or OTP.
- **Authorization** is the process of determining what actions an authenticated user is allowed to perform. It answers the question: *“What can this user do?”*
Example: A user may be allowed to SELECT data from the Patient table but not DELETE or UPDATE.

Key Difference: Authentication confirms identity, whereas authorization defines permissions and access levels. Both are essential for database security.

2. Discuss how assertions differ from triggers.

Feature	Assertion	Trigger
Scope	Applies to the entire database .	Executes on specific events : INSERT, UPDATE, DELETE.
Enforcement	Conceptual; Oracle does not implement directly.	Active; enforced automatically by the DBMS.
Level	Database-wide condition.	Row-level or statement-level.
Timing	Checked after statement execution .	Checked before or after a specific event .
Example	An instructor cannot teach more than 2 courses in a semester.	Prevent a patient from being double-booked.

Explanation:

- **Assertions** are global rules that describe constraints across tables or rows.
 - **Triggers** are procedural codes that execute automatically to enforce rules at row or table level.
-

3. Write an assertion ensuring an instructor cannot teach more than two courses in the same semester.

Schema:

Instructor(iid, iname, designation, salary)

Teaches(iid, cid, semester)

Conceptual SQL Assertion:

```

CREATE ASSERTION MaxTwoCourses
CHECK (
    NOT EXISTS (
        SELECT iid, semester
        FROM Teaches
        GROUP BY iid, semester
        HAVING COUNT(cid) > 2
    )
);

```

Explanation:

- GROUP BY iid, semester groups all courses for each instructor in a semester.
- HAVING COUNT(cid) > 2 identifies instructors teaching more than two courses.
- NOT EXISTS ensures no such violation exists in the database.

Note: Oracle cannot execute assertions directly, but this representation is **academically correct** for exams.

4. How does a trigger ensure integrity? Give an example.

Explanation:

Triggers help enforce data integrity by automatically checking or modifying data when certain events occur, preventing invalid or inconsistent data from entering the database.

Example: Prevent negative salary in the Instructor table

```

CREATE OR REPLACE TRIGGER salary_check
BEFORE INSERT OR UPDATE ON Instructor
FOR EACH ROW
BEGIN
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be negative');
    END IF;
END;
/

```

How it works:

- Before a row is inserted or updated in Instructor, the trigger checks the salary value.
- If it is negative, the trigger raises an error and stops the operation.
- This maintains **data integrity** automatically.

5. Write an assertion ensuring a patient does not visit two doctors at the same time.

Schema:

Patient(pid, pname, age, gender)

Doctor(did, dname, specialization)

Prescribe(pid, did, drugid, visit_time)

Conceptual SQL Assertion:

```
CREATE ASSERTION OneVisitAtATime
CHECK (
    NOT EXISTS (
        SELECT pid, visit_time
        FROM Prescribe
        GROUP BY pid, visit_time
        HAVING COUNT(did) > 1
    )
);
```

Explanation:

- GROUP BY pid, visit_time groups all prescriptions for each patient at the same time.
- HAVING COUNT(did) > 1 identifies patients assigned to more than one doctor simultaneously.
- NOT EXISTS ensures such conflicts are prohibited.

Trigger-based implementation (Oracle executable)

```
CREATE TRIGGER visit_check
BEFORE INSERT ON Prescribe
FOR EACH ROW
BEGIN
    IF EXISTS (SELECT 1 FROM Prescribe
               WHERE pid = :NEW.pid
               AND visit_time = :NEW.visit_time) THEN
        RAISE_APPLICATION_ERROR(-20002,'Patient already assigned');
    END IF;
END;
/
```

6. Define encryption and decryption.

- **Encryption:** The process of converting plain, readable data into an unreadable form (ciphertext) to prevent unauthorized access.
- **Decryption:** The process of converting ciphertext back into readable data (plaintext).

Use cases:

- Protecting patient medical data, passwords, bank account numbers, or confidential information.

Conceptual example:

- Plain text: HELLO
- Encrypted text: XJ2259AF..

Oracle has built-in packages like DBMS_CRYPTO for encryption and decryption.

7. Define privilege. Why are roles needed in case of privileges?

Privilege: A permission granted to a user to perform specific operations on the database or its objects.

Types:

1. **Object (Data) privileges:** SELECT, INSERT, UPDATE, DELETE on tables.
2. **System (Database) privileges:** CREATE TABLE, DROP TABLE, CREATE INDEX, ALTER TABLE.

Example:

```
GRANT SELECT ON Patient TO userA;  
GRANT INSERT ON Prescribe TO doctor_role;  
REVOKE INSERT ON Prescribe FROM doctor_role;
```

Roles:

A role is a **named collection of privileges** that can be granted to users.

Why roles are needed:

- Simplify privilege management for many users.
- Assign privileges based on job responsibilities (doctor, nurse, admin).
- Reduce errors and improve security.

- Enable cascading of privileges when roles are revoked or granted.

Example:

```
CREATE ROLE doctor_role;  
GRANT SELECT ON Patient TO doctor_role;  
GRANT INSERT, UPDATE ON Prescribe TO doctor_role;  
  
GRANT doctor_role TO dr_rahim;
```

- All privileges assigned to the role are automatically available to the user.
 - Revoking the role removes all associated privileges at once.
-